

CPSC 2010: Introduction to Computer Science

Class Recap, P vs. NP April 24, 2026

Logistics

Congratulations on making it to the last day of CPSC 2010 (and of classes)!



Final Exam: May 6, 2pm, Davies Auditorium

Practice exam will be released next Monday

Review sessions will be held by Nick (**Sat, May 2**) and Owen (**Sun, May 3**)

Nick will emphasize Racket, TMs, Boolean functions, circuits, neural nets

Owen will emphasize computer architecture (TC-201), languages, + algorithms

Jonathan's focused Racket review — last session is **today** at 3:30pm in DL 220

All late homework due before May 4, 9am

More Logistics

Congratulations on making it to the last day of CPSC 2010 (and of classes)!



I will hold office hours **today after class**

ULA applications: <https://forms.gle/v4gvam6rgsgYRRWp8> →

Apply to register your interest in becoming a ULA next fall.

(Note: Prof. Slade is teaching 2010 in the fall, so the course will be slightly different, but you should still be qualified to ULA! To ULA with me, apply next spring. You can do either/both.)



What have we learned this semester?

1900-1930

What even *is* computation?

Are there limits to what can be computed?

Hilbert's problems

Hilbert's 10th problem (1900):

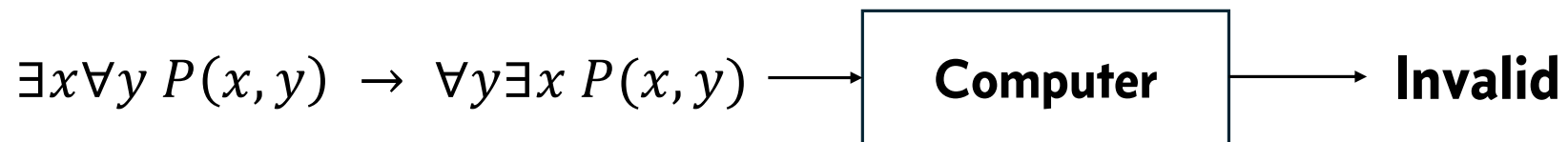
Given a polynomial equation with integer coefficients, **devise a process according to which it can be determined in a finite number of operations** whether the equation has an integer solution.



Hilbert's problems

Entscheidungsproblem (1928):

Is there a **finite, purely mechanical procedure** that, given any formula of first-order logic, determines whether that formula is logically valid?



Hilbert's problems



$$3x^2 - 2xy - y^2z - 7 = 0 \rightarrow \boxed{\text{Computer}} \rightarrow x = 1, y = 2, z = -2$$

Finite

Instructions



What instructions should
be allowed in an algorithm?

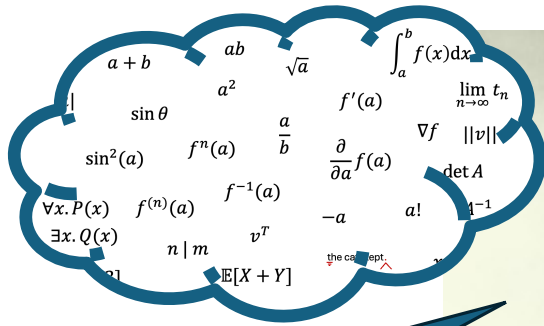
Any answer to this question defines a
“programming language”

A **good** answer needs to be:

Simple

General

Two approaches to defining a minimal language



What are functions?
How do we write down a
function, and how do we
determine what it does on
any given input?



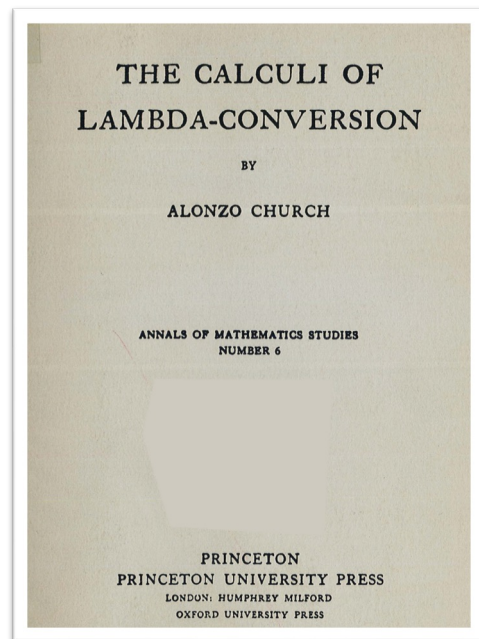
Alonzo Church



What kinds of instructions
can human computers
actually understand and
follow?



Alan Turing

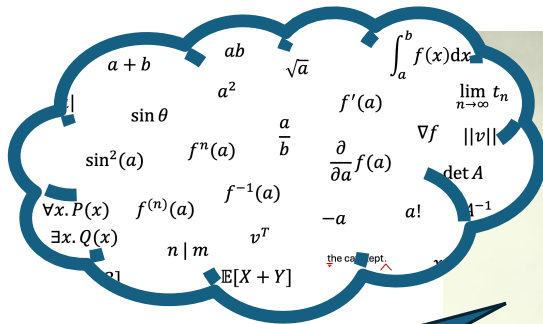


Chapter I

INTRODUCTORY

1. **THE CONCEPT OF A FUNCTION.** Underlying the formal calculi which we shall develop is the concept of a function, as it appears in various branches of mathematics, either under that name or under one of the synonymous names, "operation" or "transformation." The study of the general properties of functions, independently of their appearance in any particular mathematical

What is a function?



What are functions?
How do we write down a
function, and how do we
determine what it does on
any given input?



Alonzo Church

Church's answer: a
function is a **lambda term**.

$$f = \lambda x. x$$

$$g = \lambda x. x + (x * 2)$$

$$h = \lambda x. (f (g x))$$

To evaluate `<lit>`:

- 1 Return the value itself.

To evaluate `<id>`:

- 1 Look up the identifier in the active environment.

 **Warning:** If not found, raise `<id>` **undefined error**.

To evaluate (`<expr>` `<expr>`...):

- 1 Find the value of the procedure (first expression).

 **Warning:** If the value of the first expression is not a procedure, raise **application: not a procedure error**.

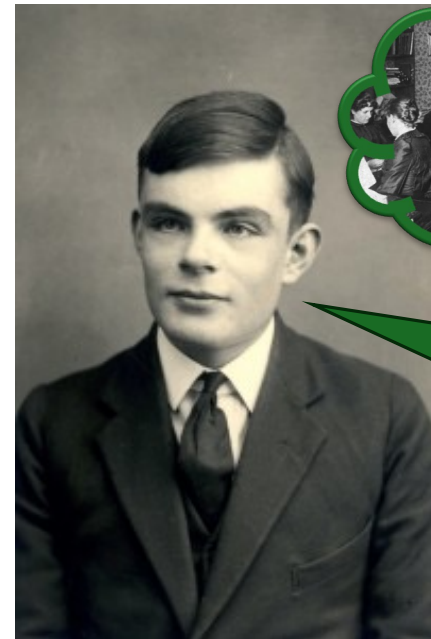
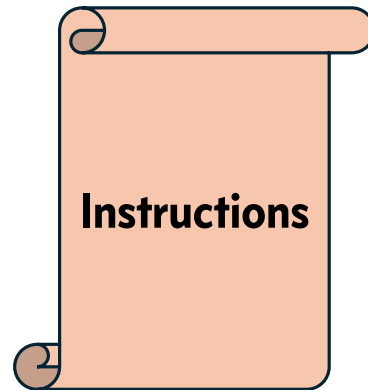
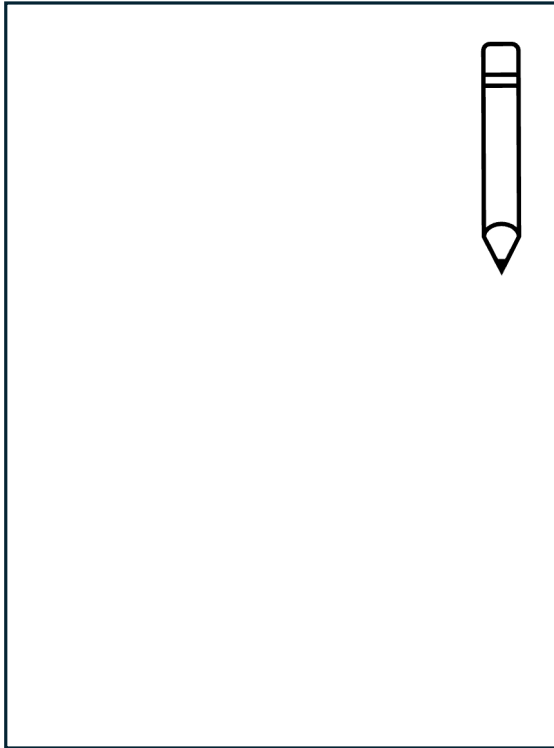
- 2 Find the values of the argument expressions.
- 3 Call the procedure on the arguments; return the result.

SPECIAL FORMS (EXCEPTIONS TO RULE 3)

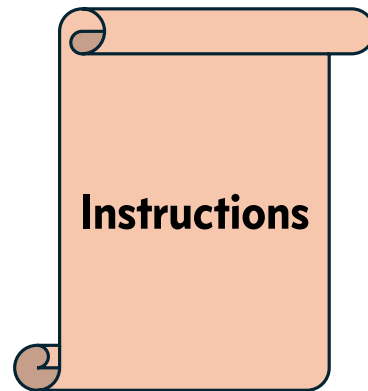
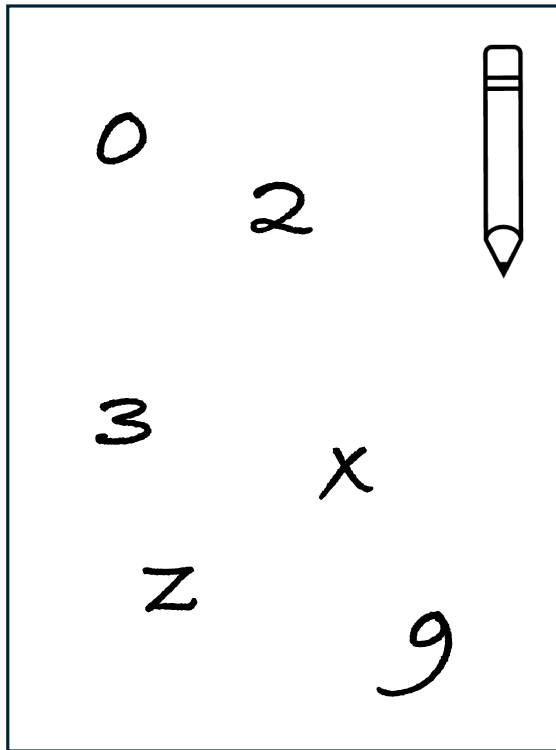
To evaluate (`define` `<id>` `<expr>`):

- 1 Evaluate the expression.
- 2 Bind the identifier to the value of the expression in the active environment.

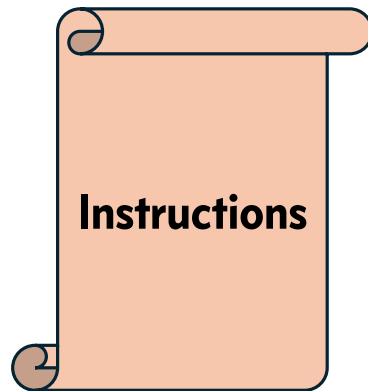
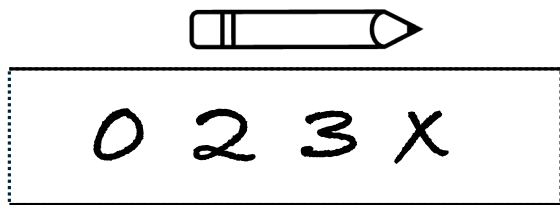
... AND MANY MORE ...



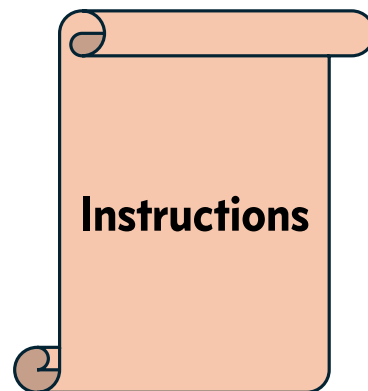
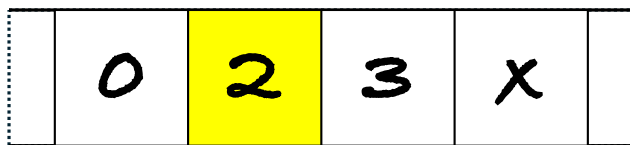
What kinds of instructions
can human computers
actually understand and
follow?



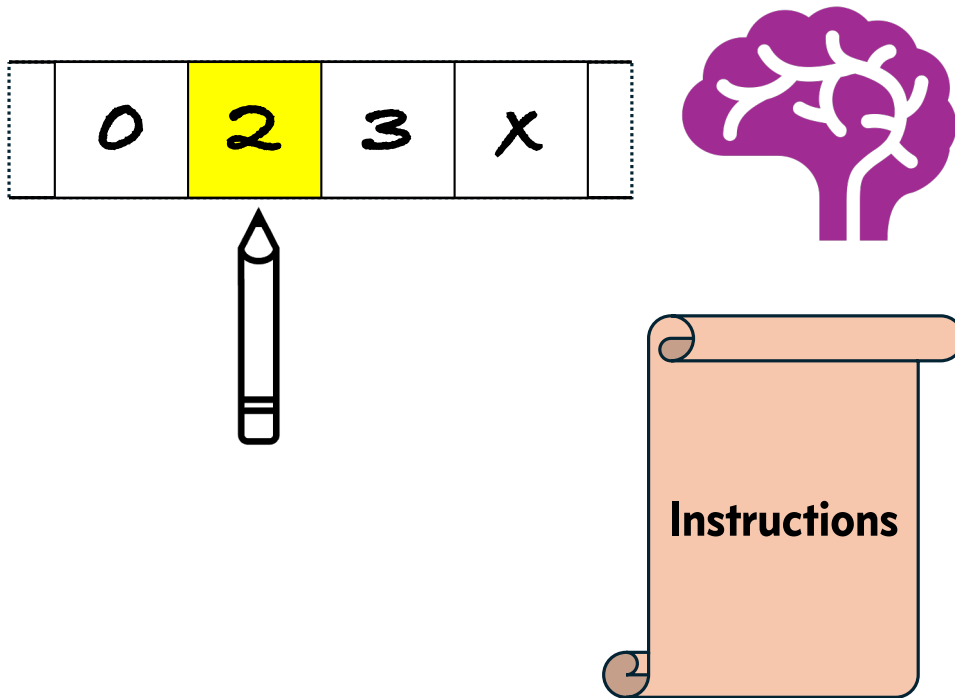
Simplification 1: On the paper, we write **symbols** from a finite alphabet.



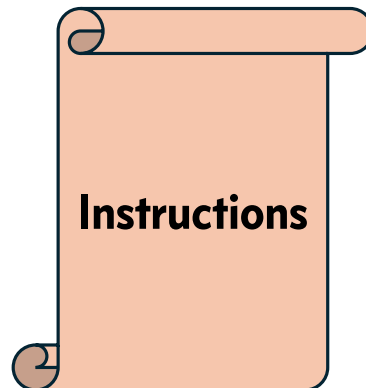
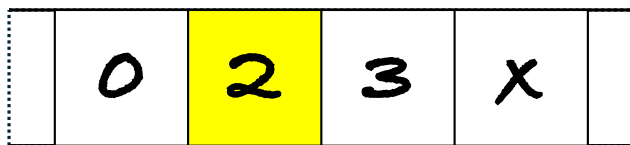
Simplification 2: Let's say the paper is organized as a long **1D tape**. (*Infinitely* long, because you can always buy more paper.)



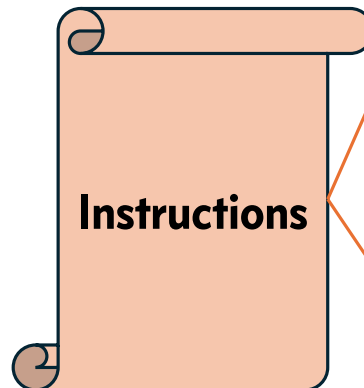
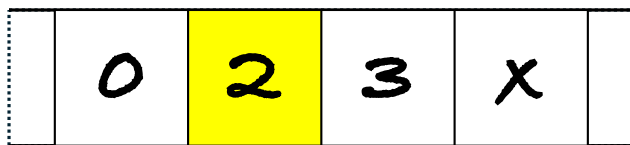
Simplification 3: The tape is ruled into **cells**, each of which holds one symbol. At any moment, your eyes can only focus on **one** cell.



Simplification 4: Your brain can only remember a finite amount of information (which is why you need the paper in the first place).



Simplification 4: Your brain can only remember a finite amount of information (which is why you need the paper in the first place).



Available actions:

Look left (or right)
Erase a symbol / write a new symbol
Change your state of mind

Available inputs when deciding what to do:

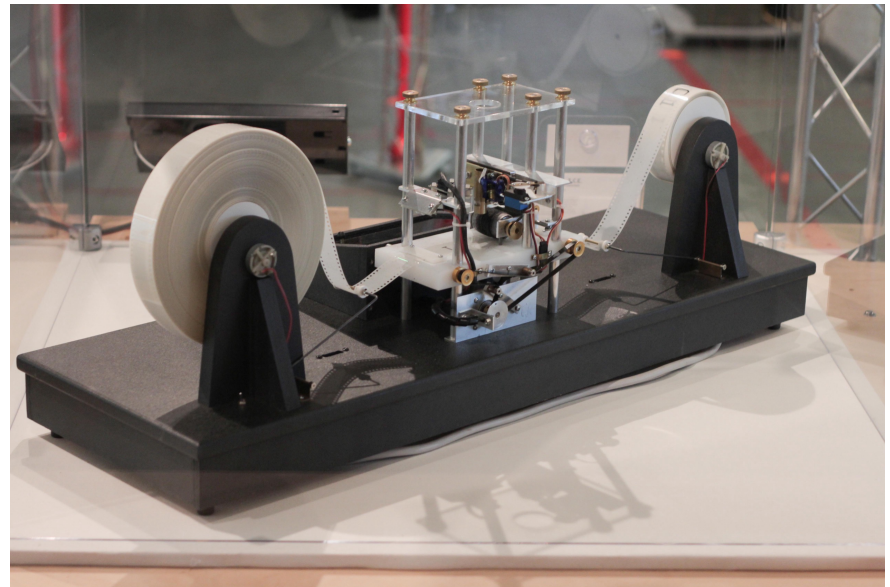
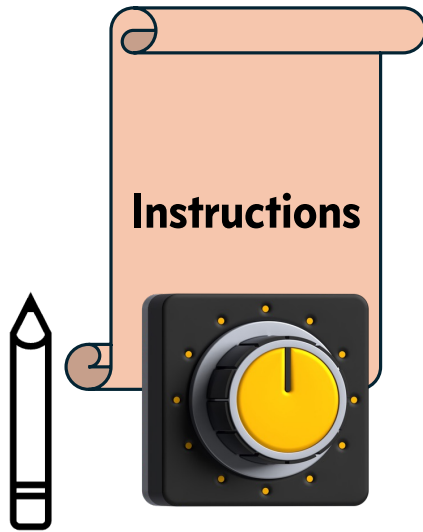
Current state of mind
Current symbol you are looking at

So an "instruction" looks like

If you are in **state-of-mind** 7,
and you **see the symbol** "x",
change to state-of-mind 32,
replace the symbol "x" with a "y",
and **look right** one cell.

Turing machines

If you can frame an algorithm this way...



...you could imagine a machine that executes the algorithm.

Note: by “Turing machine,” we mean a particular set of instructions, from which we build a particular machine.

Computability

A function $f : \{0,1\}^* \rightarrow \{0,1\}^*$ is **computable** if there exists a **Turing machine** that, when run with x on its input tape, halts with $f(x)$ on its output tape, for all inputs x .

The halting problem

- The Halting Problem

SCOOPING THE LOOP SNOOPER

A proof that the Halting Problem is undecidable

Geoffrey K. Pullum

(School of Philosophy, Psychology and Language Sciences, University of Edinburgh)

- Note that

No general procedure for bug checks will do.

Now, I won't just assert that, I'll prove it to you.

I will prove that although you might work till you drop,
you cannot tell if computation will stop.

tion

alts

machine.

1900-1930

What even *is* computation?

Are there limits to what can be computed?

1940s

How can physical things compute?

(brains, electronic machines)

1850s

What are the laws governing reasoning?

*G. Boole
Quincy Coll
Cock*

AN INVESTIGATION
OF
THE LAWS OF THOUGHT,
ON WHICH ARE FOUNDED
THE MATHEMATICAL THEORIES OF LOGIC
AND PROBABILITIES.

Barbari (AAI-1) [\[edit \]](#)

All men are mortal. (MaP)
All Greeks are men and some Greeks
exist (SaM)
∴ Some Greeks are mortal. (SiP)

Celaront (EAO-1) [\[edit \]](#)

Similar: *Cesaro (EAO-2)*
No reptiles have fur. (MeP)
All snakes are reptiles. (SaM)
∴ Some snakes have no fur. (SoP)

Camestros (AEO-2) [\[edit \]](#)

Similar: *Calemos (AEO-4)*
All horses have hooves. (PaM)
No humans have hooves. (SeM)
∴ Some humans are not horses. (SoP)

Darii (AII-1) [\[edit \]](#)

Similar: *Datisi (AII-3)*
All rabbits have fur. (MaP)
Some pets are rabbits. (SiM)
∴ Some pets have fur. (SiP)

Barbara (AAA-1) [\[edit \]](#)

All men are mortal. (MaP)
All Greeks are men. (SaM)
∴ All Greeks are mortal. (SaP)

Bocardo (OAO-3) [\[edit \]](#)

Some cats are not pets. (MoP)
All cats are mammals. (MaS)
∴ Some mammals are not pets. (SoP)

Baroco (AOO-2) [\[edit \]](#)

All cats are mammals. (PaM)
Some pets are not mammals. (SoM)
∴ Some pets are not cats. (SoP)

Celarent (EAE-1) [\[edit \]](#)

Similar: *Cesare (EAE-2)*
No reptile has fur. (MeP)
All snakes are reptiles. (SaM)
∴ No snake has fur. (SeP)

Ferio (EIO-1) [\[edit \]](#)

Similar: *Festino (EIO-2)*, *Ferison (EIO-3)*,
Fresison (EIO-4)
No homework is fun. (MeP)
Some reading is homework. (SiM)
∴ Some reading is not fun. (SoP)

Boole: Can we reduce deduction to calculation?

Key idea: let **variables** stand for **propositions**, not just numbers

Let P represent "It is raining"

Let Q represent "I have an umbrella"

Let R represent "I will get wet"

$$R = P * (1 - Q)$$

[I will get wet if and only if it is raining and I have no umbrella]

$$(1 - P) + Q = 1$$

[It is true that either it isn't raining or I have an umbrella.]

1940s

How can physical things compute?

(brains, electronic machines)

Neural
Networks



Boolean
Logic



Electrical
Circuits

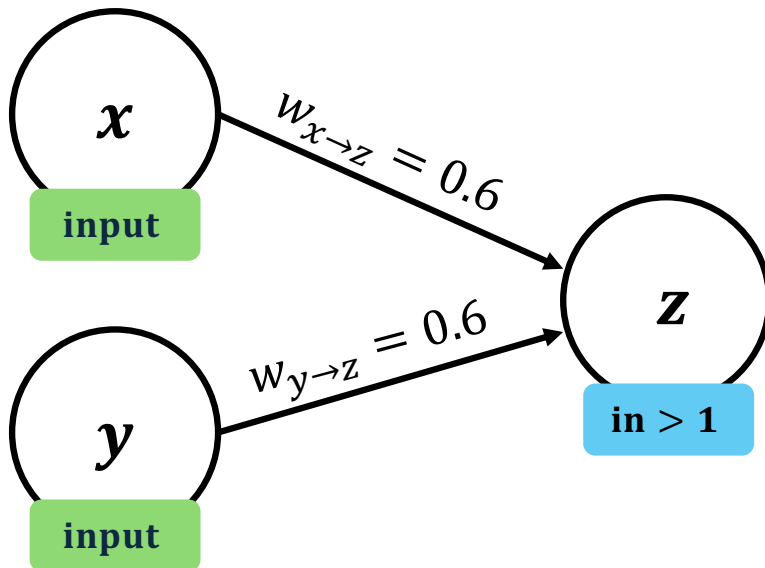
A LOGICAL CALCULUS OF THE IDEAS IMMANENT IN NERVOUS ACTIVITY*

■ **WARREN S. MCCULLOCH AND WALTER PITTS**
University of Illinois, College of Medicine,
Department of Psychiatry at the Illinois Neuropsychiatric Institute,
University of Chicago, Chicago, U.S.A.

Because of the “all-or-none” character of nervous activity, neural events and the relations among them can be treated by means of propositional logic. It is found that the behavior of every net can be described in these terms, with the addition of more complicated logical means for nets containing circles; and that for any logical expression satisfying certain conditions, one can find a net behaving in the fashion it describes. It is shown that many particular choices among possible

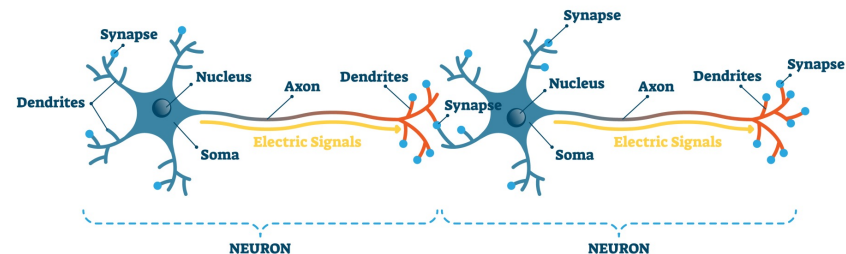
Neural networks

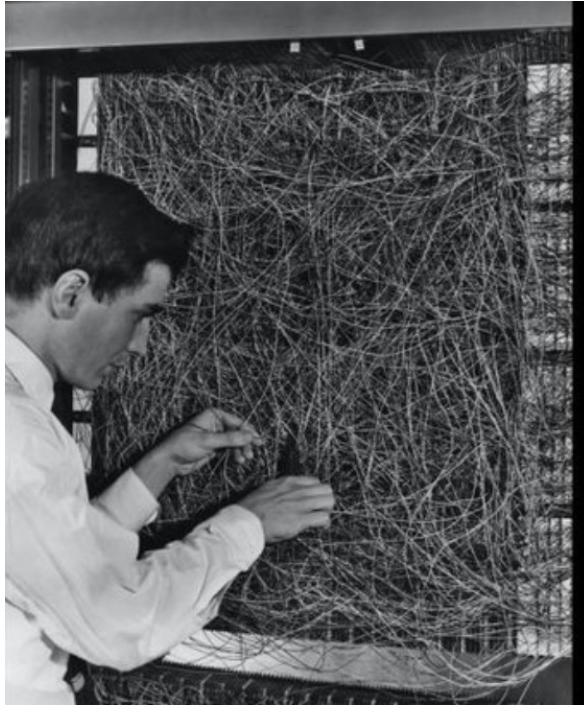
A **neural network** is a collection of **neurons** connected by **synapses**



A neuron's **input signal** is the sum of the incoming signal on each synapse

If it exceeds a threshold, the neuron fires.





Electronic 'Brain' Teaches Itself

The Navy last week demonstrated the embryo of an electronic computer named the Perceptron which, when completed in about a year, is expected to be the first non-living mechanism able to "perceive, recognize and identify its surroundings without human training or control." Navy officers demonstrating a preliminary form of the device in Washington said they hesitated to call it a machine because it is so much like a "human being without life."

Dr. Frank Rosenblatt, research psychologist at the Cornell Aeronautical Laboratory, Inc., Buffalo, N. Y., designer of the Perceptron, conducted the demonstration. The machine, he said, would be the first electronic device to think as the human brain. Like humans, Perceptron will make mistakes at first, "but it will grow wiser as it gains experience," he said.

The first Perceptron, to cost about \$100,000, will have about 1,000 electronic "association cells" receiving electrical impulses from an eyelike scanning device with 400 photocells. The human brain has ten billion responsive cells, including 100,000,000 connections with the eye.

Difference Recognized

The concept of the Perceptron was demonstrated on the Weather Bureau's \$2,000,000 IBM 704 computer. In one experiment, the 704 computer was shown 100 squares situated at random either on the

recognize the difference between right and left, almost the way a child learns.

When fully developed, the Perceptron will be designed to remember images and information it has perceived itself, whereas ordinary computers remember only what is fed into them on punch cards or magnetic tape.

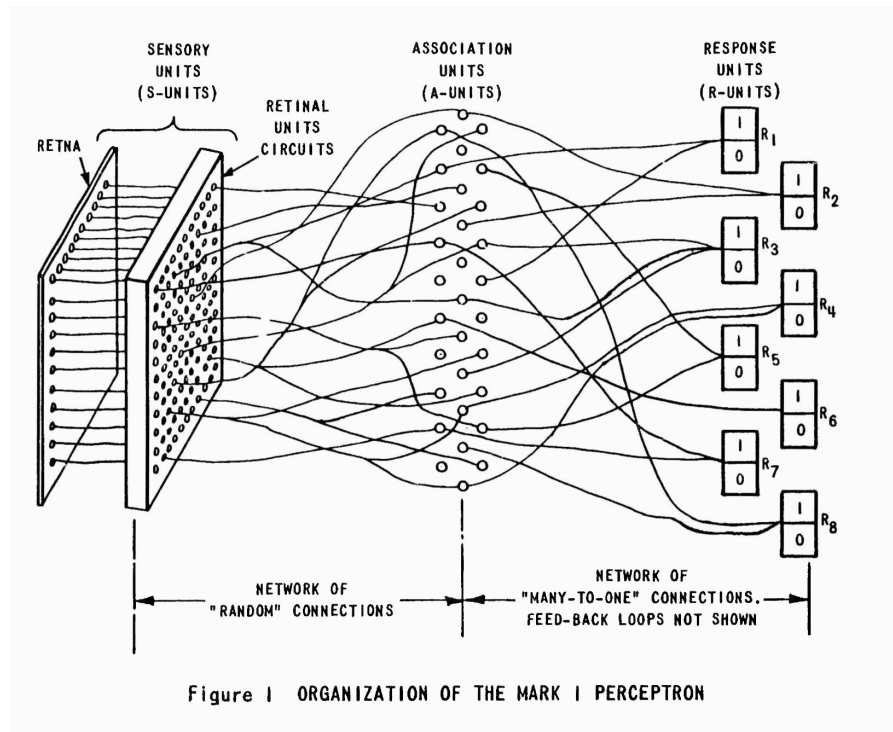
Later Perceptrons, Dr. Rosenblatt said, will be able to recognize people and call out their names. Printed pages, longhand letters and even speech commands are within its reach. Only one more step of development, a difficult step, he said, is needed for the device to hear speech in one language and instantly translate it to speech or writing in another language.

Self-Reproduction

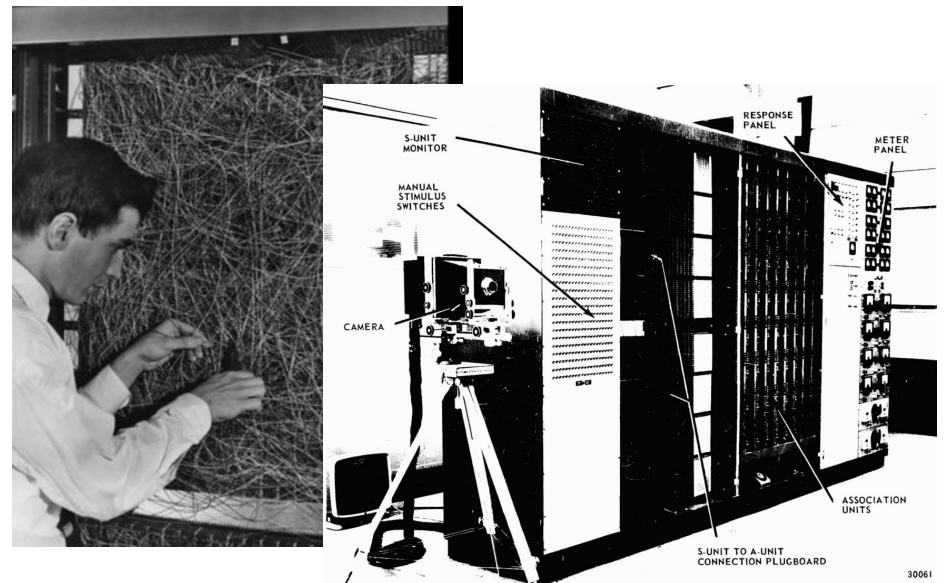
In principle, Dr. Rosenblatt said, it would be possible to build Perceptrons that could reproduce themselves on an assembly line and which would be "conscious" of their existence.

Perceptron, it was pointed out, needs no "priming." It is not necessary to introduce it to surroundings and circumstances, record the data involved and then store them for future comparison as is the case with present "mechanical brains." It literally teaches itself to recognize objects the first time it encounters them. It uses a camera-eye lens to scan objects or survey situations, and an electrical impulse

The Mark I Perceptron

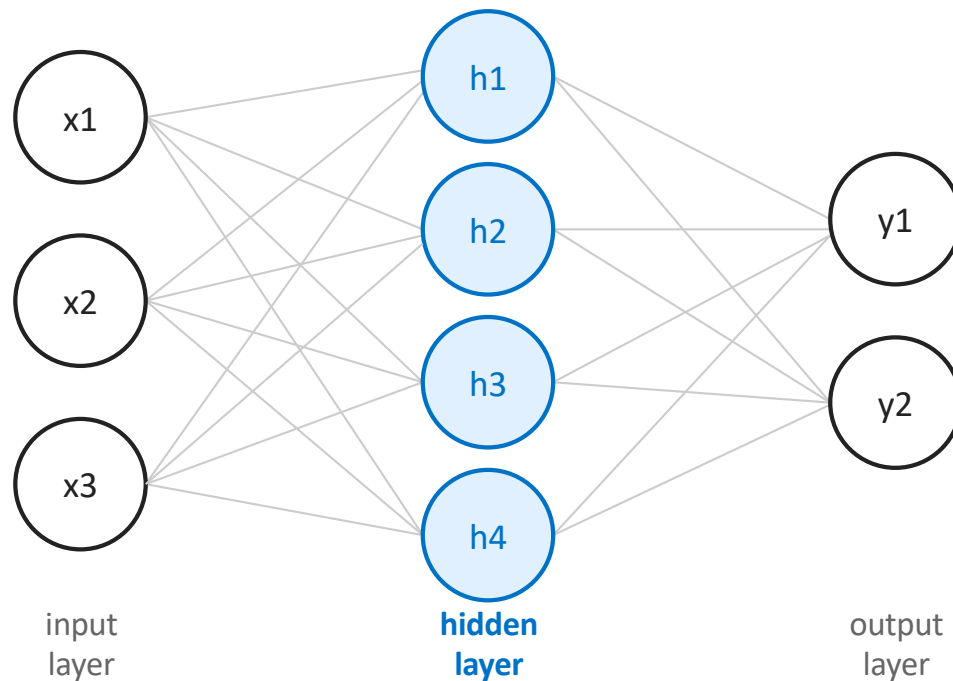


Built at Cornell Aeronautical Laboratory



The Multi-Layer Perceptron (MLP)

Solution: stack layers of perceptrons.



Hidden layers let the network learn intermediate features

An MLP with even one hidden layer can approximate **any** function (Universal Approximation Theorem)

This is what modern **deep learning** is built on: many layers of simple perceptron-like units

Training deep nets: gradient descent

Problem: How do we adjust weights in a multi-layer network? The original perceptron rule only works for one layer.

Idea: define a **loss function** measuring how wrong the network is, then nudge *all* weights to shrink it.

Finite-differences gradient descent (the simple version)

For each weight w :

1. Try increasing w by a tiny amount ϵ
2. Measure: $\Delta\text{loss} = \text{loss}(w+\epsilon) - \text{loss}(w)$
3. If $\Delta\text{loss} < 0$: increase w ; else decrease w
(Update: $w \leftarrow w - \alpha \times (\Delta\text{loss} / \epsilon)$)

This is slow (try each weight one at a time!) but conceptually simple.

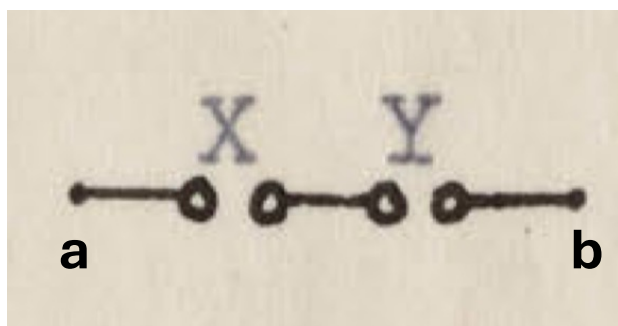
Backpropagation (Rumelhart, Hinton & Williams, 1986) computes all gradients at once using the chain rule — making deep learning practical.

*E. E.
Shannon*

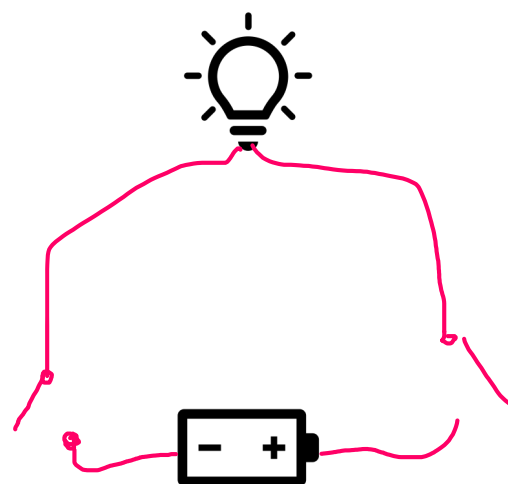
A SYMBOLIC ANALYSIS
1948
OF
RELAY AND SWITCHING CIRCUITS

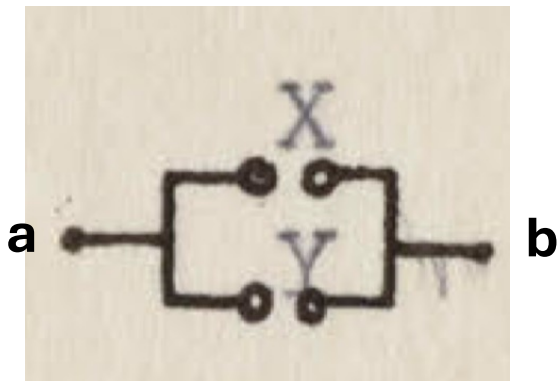
by

Claude Elwood Shannon

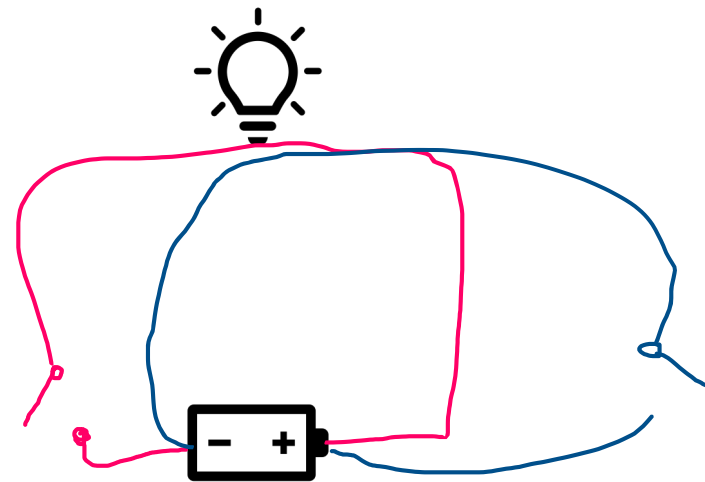


$$X_{ab} = X \cdot Y$$

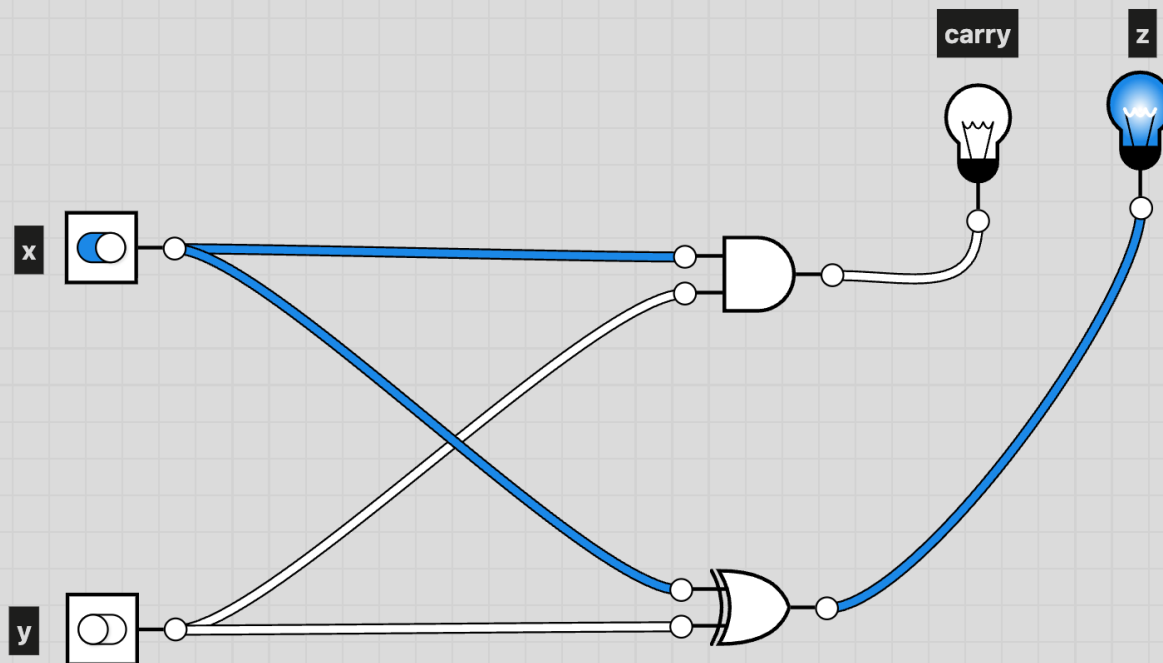


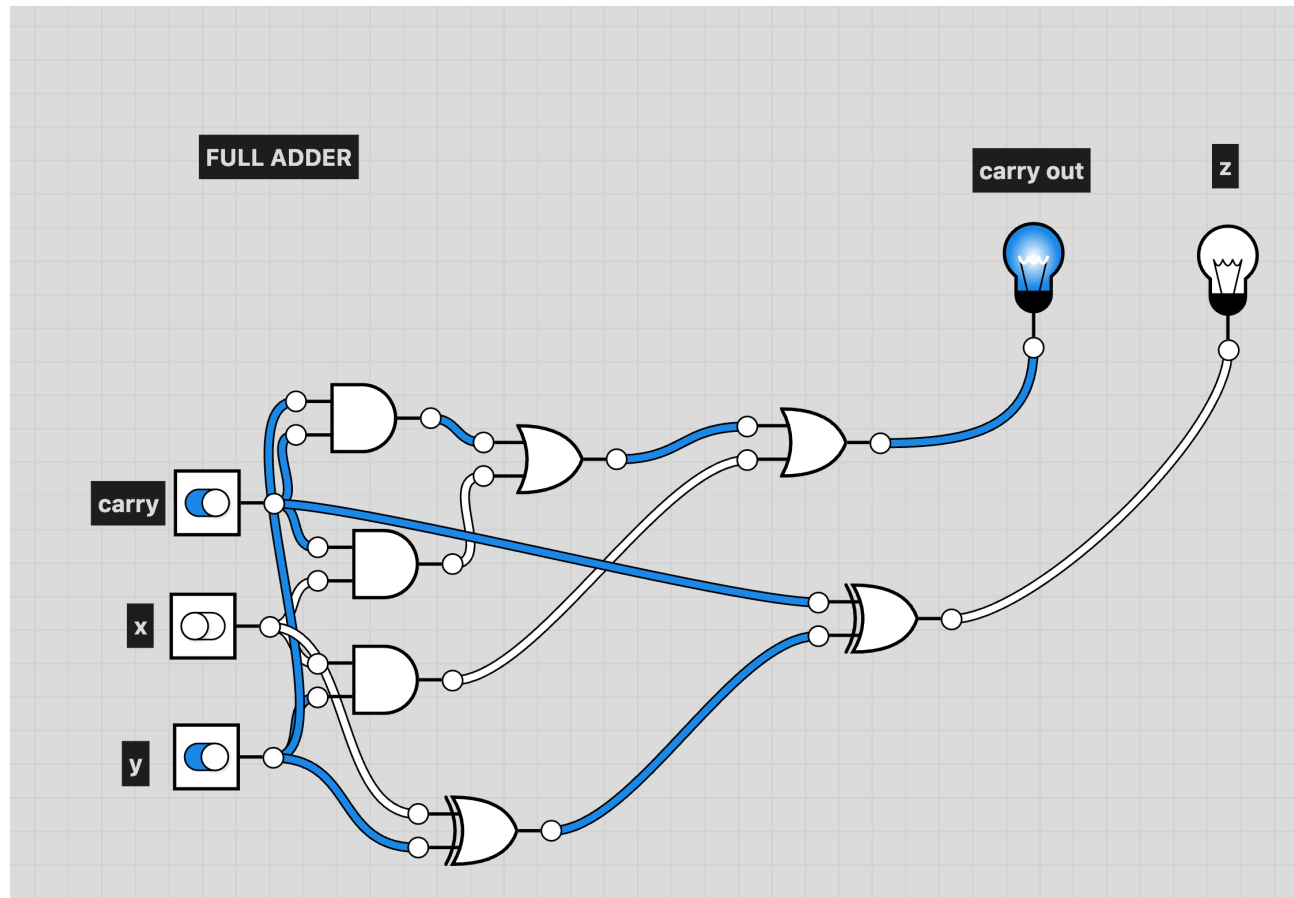


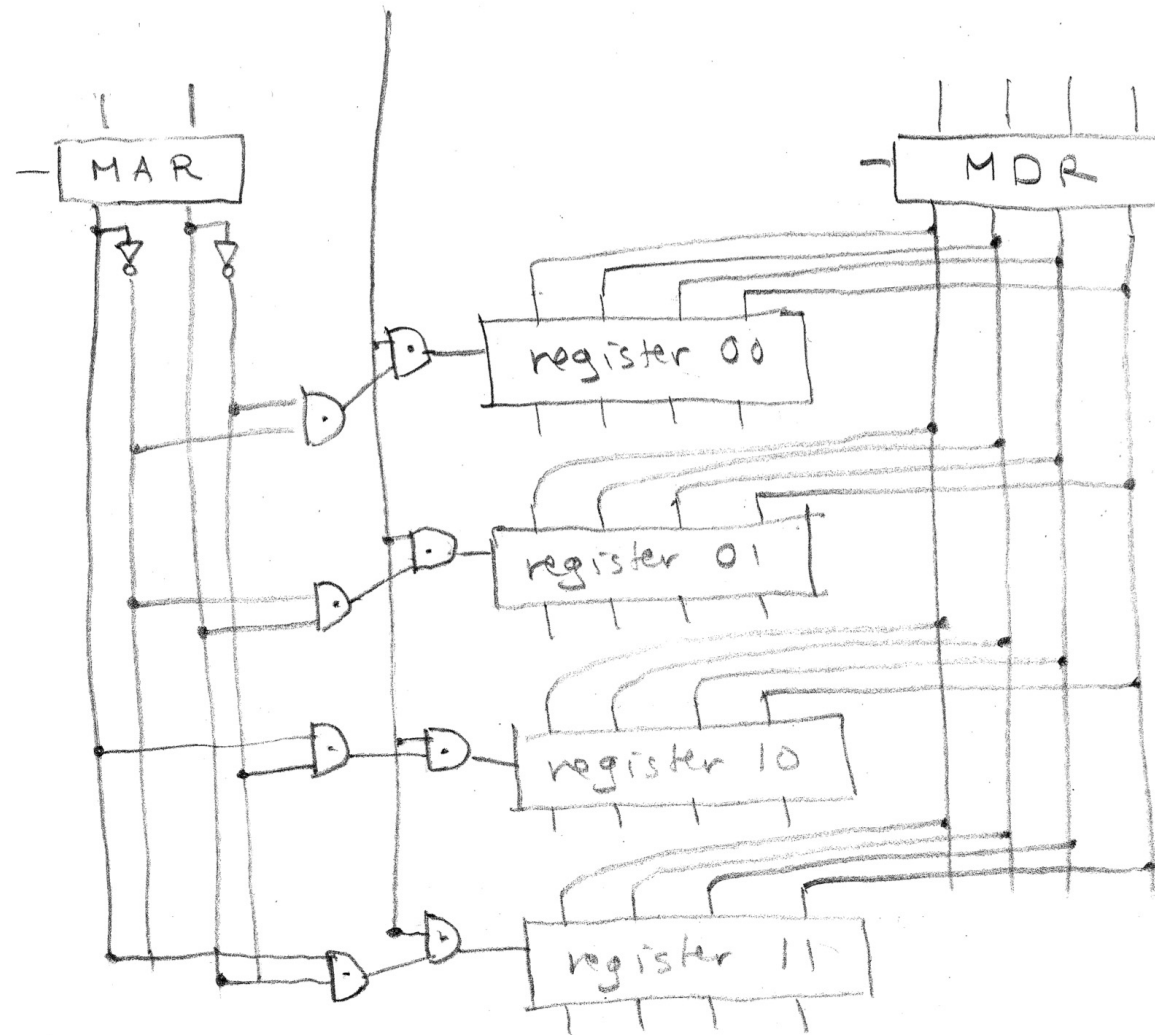
$$X_{ab} = X + Y$$

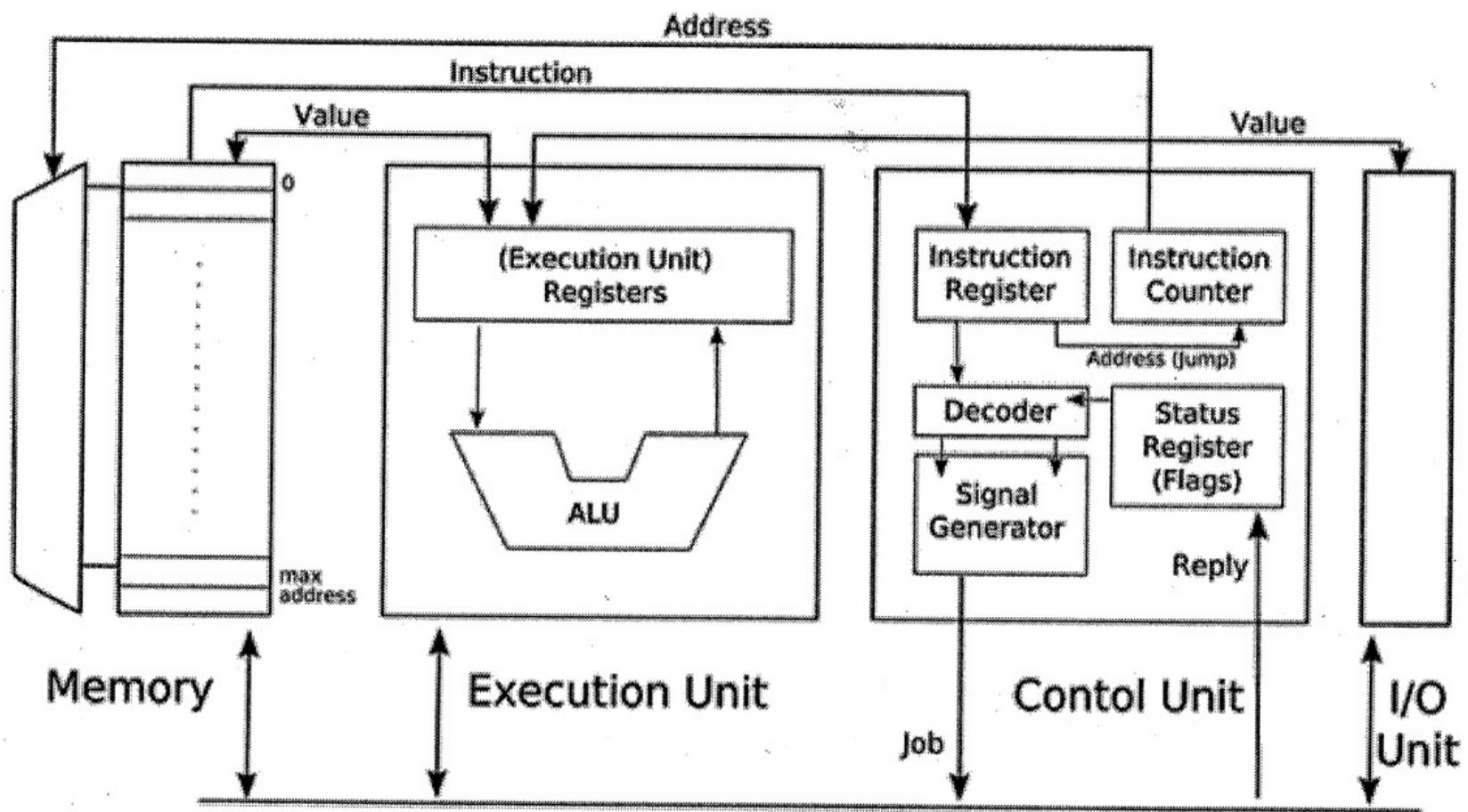


HALF-ADDER









Assembly Program

Example: Sum input until zero

Load

Assemble

```
0 ; Reads numbers until 0, prints the sum
1 start: input
2 skipzero
3 jump add_to_total
4 jump print_and_halt
5 add_to_total: add total
6 store total
7 jump start
8 print_and_halt: load total
9 output
10 halt
11 total: data 0
```

Input values (space/comma/newline separated): e.g. 14, -5, 10, 0

Load Inputs

Assembled 11 words. Labels: start=0, add_to_total=4, print_and_halt=7, total=10

Execution

Reset

Step

Run 100

Stop

PC

0

ACC (signed)

0

ACC (bin)

0000 0000 0000
0000

AEB

0

Current Instruction

0101 0000 0000
0000 | input

Step Count

0

Last Event

ready

Halted

false

Show memory near address: 0

Go

Addr	Word (bin)	Signed	Decoded
0	0101 0000 0000 0000	20480	input
1	1000 0000 0000 0000	-32768	skipzero
2	0111 0000 0000 0100	28676	jump 4
3	0111 0000 0000 0111	28679	jump 7
4	0011 0000 0000 1010	12298	add 10
5	0010 0000 0000 1010	8202	store 10
6	0111 0000 0000 0000	28672	jump 0
7	0001 0000 0000 1010	4106	load 10
8	0110 0000 0000 0000	24576	output
9	0000 0000 0000 0000	0	halt
10	0000 0000 0000 0000	0	halt
11	0000 0000 0000 0000	0	halt
12	0000 0000 0000 0000	0	halt

Input Queue

(empty)

Output

(no output)

1950s

What patterns can machines recognize?

What kinds of patterns show up in **language**?

Languages

Could you write programs in Racket, TC-201, TM to *decide* L?

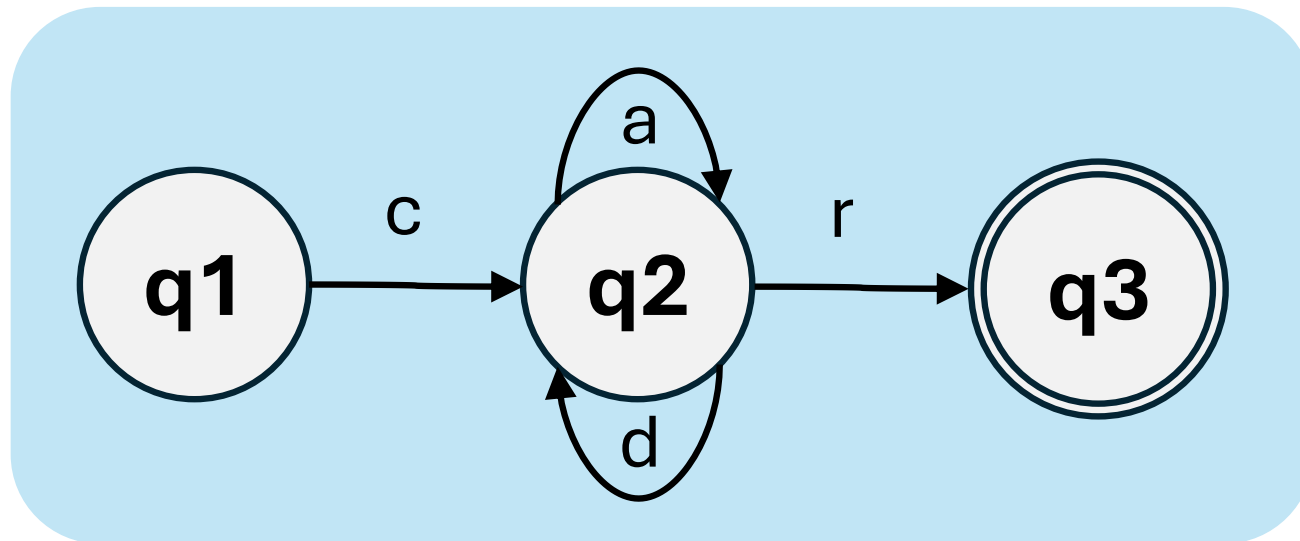
$$L_1 = \{car, cdr, cadr, cddr, caar, caddr, cdadr, \dots\}$$

$$L_2 = \{0, 10, 100, 110, 1000, 1010, 1100, 1110, \dots\}$$

$$L_3 = \{aardvark, abaci, aback, abacus, abandon, abase, \dots\}$$

$$L_4 = \{ M \mid M \text{ is a Turing machine that} \\ \text{halts when run on empty tape} \}$$

Deterministic finite automata (DFAs)



REPRESENTATION OF EVENTS IN NERVE NETS AND FINITE AUTOMATA*

S. C. Kleene

1 Stimuli and Response

An organism or an automaton receives stimuli via its sensory receptor organs, and performs actions via its effector organs. To say that certain actions are a response to certain stimuli means, in the simplest case, that the actions are performed when and only when those stimuli occur.

In the general case both the stimuli and the actions may be very complicated.

In order to simplify the analysis, we may begin by leaving out of account the complexities of the response. We reason that any sort of stimulation, or briefly any event, which affects action in the sense that different actions ensue according as the event occurs or not, under some set of other circumstances held fixed, must have a representation in the state of the organism or automaton, after the event has occurred and prior to the ensuing action.

So we ask what kind of events are capable of being represented in the state of an automaton.

Regular Expressions

Theorem: For any regular expression e , $L(e)$ is a regular language.

Proof: Single letters can be recognized by a very simple DFA. Unions and Kleene closures of regular languages are regular.

Natural languages, formally

A **grammar** for a (natural) language precisely specifies which sentences are grammatical.

- (1) John ate a sandwich
- (2) Sandwich a ate John.

Natural languages, formally

Goal:

generate **all** and **only** the grammatical English sentences

Can we do this with a **regular** language?

(i.e., a regular expression or DFA?)

THE REDUNDANCY OF ENGLISH

CLAUDE E. SHANNON

*Bell Laboratories,
Murray Hill, N. J.*

- | | | |
|----|--|---------------------------------------|
| 1. | XFOML RXKHRJFFJUJ ZLPWCFWKCYJ FFJEYVKCQSGHYD | uniform at random (characters) |
| 2. | OCRO HLI RGWR NMIELWis EU LL NBNeseBYA TH EEI | unigram character frequencies |
| 3. | ON IE ANTSOUTINYS ARE T INCTORE ST BE S DEAMY ACHIN D ILONASIVE TUcoowe AT TEASONARE FUSO | bigram character frequencies |
| 4. | IN NO IST LAT WHEY CRATICT FROURE BIRS GROCID PONDENOME OF DEMONSTURES OF THE RETAGIN IS REGI ACTIONA OF CRE. | trigram character frequencies |
| 5. | REPRESENTING AND SPEEDILY IS AN GOOD APT OR COME CAN DIFFERENT NATURAL HERE HE THE A IN CAME THE TO OF TO EXPERT GRAY COME TO FURNISHES THE LINE MESSAGE HAD BE THESE. | unigram word frequencies |
| 6. | THE HEAD AND IN FRONTAL ATTACK ON AN ENGLISH WRITER THAT THE CHARACTER OF THIS POINT IS THEREFORE ANOTHER METHOD FOR THE LETTERS THAT THE TIME OF WHO EVER TOLD THE PROBLEM FOR AN UNEXPECTED. | bigram word frequencies |

Natural languages, formally

THREE MODELS FOR THE DESCRIPTION OF LANGUAGE*

Noam Chomsky

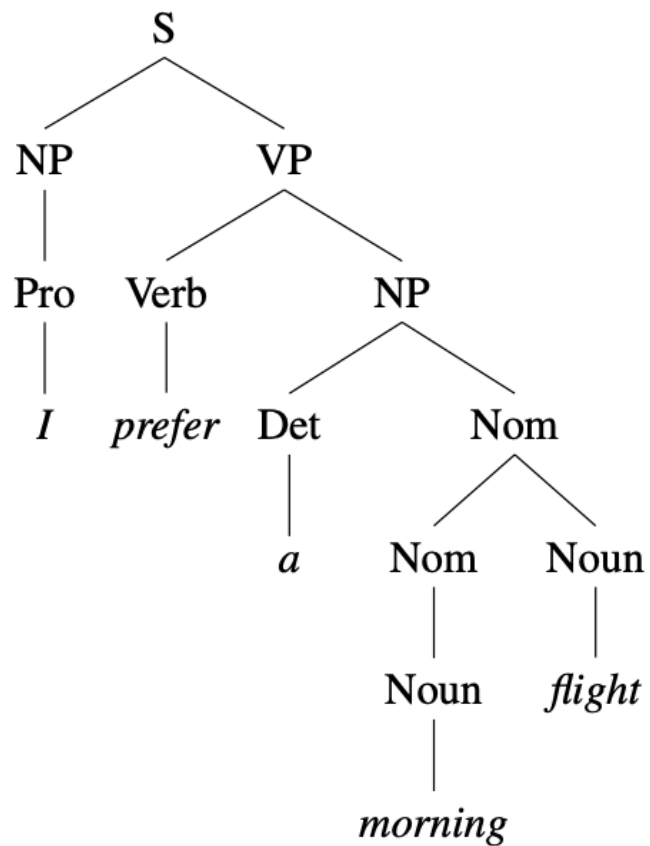
Department of Modern Languages and Research Laboratory of Electronics
Massachusetts Institute of Technology
Cambridge, Massachusetts

[The cat] ran

[The cat [the dog] chased] ran

[The cat [the dog [the man] owned] chased] ran

[The cat [the dog [the man [I] talked to yesterday] owned] chased] ran



Grammar Rules	Examples
$S \rightarrow NP VP$	I + want a morning flight
$NP \rightarrow$ <i>Pronoun</i> $\quad \quad \quad \mid$ <i>Proper-Noun</i> $\quad \quad \quad \mid$ <i>Det Nominal</i>	I Los Angeles a + flight
$Nominal \rightarrow Nominal Noun$	morning + flight
$\quad \quad \quad \mid$ <i>Noun</i>	flights
$VP \rightarrow$ <i>Verb</i>	do
$\quad \quad \quad \mid$ <i>Verb NP</i>	want + a flight
$\quad \quad \quad \mid$ <i>Verb NP PP</i>	leave + Boston + in the morning
$\quad \quad \quad \mid$ <i>Verb PP</i>	leaving + on Thursday
$PP \rightarrow Preposition NP$	from + Los Angeles
$Noun \rightarrow$ <i>flights</i> <i>flight</i> <i>breeze</i> <i>trip</i> <i>morning</i> $Verb \rightarrow$ <i>is</i> <i>prefer</i> <i>like</i> <i>need</i> <i>want</i> <i>fly</i> <i>do</i> $Adjective \rightarrow$ <i>cheapest</i> <i>non-stop</i> <i>first</i> <i>latest</i> $\quad \quad \quad \mid$ <i>other</i> <i>direct</i> $Pronoun \rightarrow$ <i>me</i> <i>I</i> <i>you</i> <i>it</i> $Proper-Noun \rightarrow$ <i>Alaska</i> <i>Baltimore</i> <i>Los Angeles</i> $\quad \quad \quad \mid$ <i>Chicago</i> <i>United</i> <i>American</i> $Determiner \rightarrow$ <i>the</i> <i>a</i> <i>an</i> <i>this</i> <i>these</i> <i>that</i> $Preposition \rightarrow$ <i>from</i> <i>to</i> <i>on</i> <i>near</i> <i>in</i> $Conjunction \rightarrow$ <i>and</i> <i>or</i> <i>but</i>	

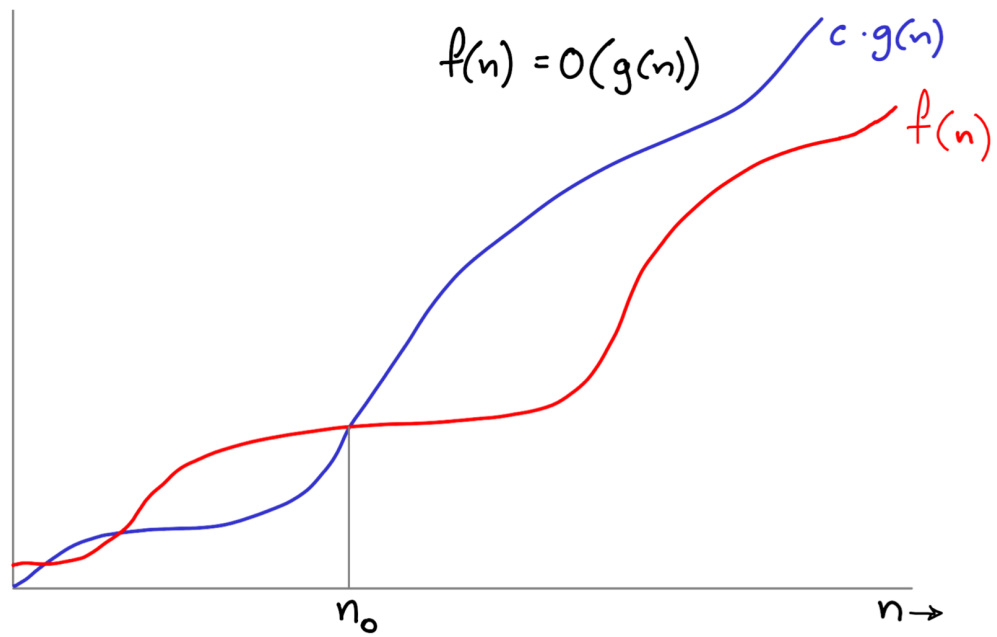
```

Goal ::= MainClass ( ClassDeclaration )* <EOF>
MainClass ::= "class" Identifier "{" "public" "static" "void" "main" "(" "String" "[" "]" Identifier ")" "{" Statement "}" "}"
ClassDeclaration ::= "class" Identifier ("extends" Identifier)? "{" ( VarDeclaration )* ( MethodDeclaration )* "}"
VarDeclaration ::= Type Identifier ","
MethodDeclaration ::= "public" Type Identifier "(" ( Type Identifier ( "," Type Identifier )* )? ")" "{" ( VarDeclaration )* ( Statement )* "return" Expression ";" "}"
Type ::= "int" "[" "]"
      | "boolean"
      | "int"
      | Identifier
Statement ::= "{" ( Statement )* "}"
      | "if" "(" Expression ")" Statement "else" Statement
      | "while" "(" Expression ")" Statement
      | "System.out.println" "(" Expression ")" ";"
      | Identifier "=" Expression ";"
      | Identifier "[" Expression "]" "=" Expression ";"
Expression ::= Expression ( "&&" | "<" | "+" | "-" | "*" ) Expression
      | Expression "[" Expression "]"
      | Expression "." "length"
      | Expression "." Identifier "(" ( Expression ( "," Expression )* )? ")"
      | <INTEGER_LITERAL>
      | "true"
      | "false"
      | Identifier
      | "this"
      | "new" "int" "[" Expression "]"
      | "new" Identifier "(" " )"
      | "!" Expression
      | "(" Expression ")"
Identifier ::= <IDENTIFIER>

```


1960s

What can be computed efficiently?



1970s

What cannot be computed efficiently?